

Java 期末考试复习重点总结

面向大一期末，涵盖基本语法、面向对象、常用类、异常、集合、I/O 六大模块

一、Java 基本语法

1.1 基本数据类型

类型	关键字	字节数	默认值
字节型	byte	1	0
短整型	short	2	0
整型	int	4	0
长整型	long	8	0L
单精度浮点	float	4	0.0f
双精度浮点	double	8	0.0d
字符型	char	2	'?'
布尔型	boolean	未定	false

易错点：

- long 型赋值需加 L：long a = 100L;
- float 型赋值需加 f：float b = 3.14f;
- char 在 Java 中占 **2 字节**（Unicode 编码，不是 C 语言的 1 字节）

1.2 类型转换

```
// 自动类型转换（小→大）：byte→short→int→long→float→double
int a = 10;
double b = a;           // 自动转换

// 强制类型转换（大→小）：可能丢失精度
double c = 3.14;
int d = (int) c;        // d = 3，小数部分被截断
```

1.3 运算符（易错点）

运算符	说明	注意事项
/	除法	整数相除结果仍为整数：5/2 = 2，不是 2.5
%	取余	5%2 = 1，结果的符号与被除数相同
&& /	短路与/或	短路特性：左边能决定结果时右边不执行
& /	非短路与/或	两边都会执行
++ / --	自增/自减	i++ 先取值后自增，++i 先自增后取值
==	相等比较	基本类型比值，引用类型比地址（重要！）

短路示例（高频考题）：

```
int x = 5;
boolean flag = (x > 5) && (++x > 5); // ++x 不执行，x 仍为 5
boolean flag2 = (x > 5) & (++x > 5); // & 不会短路，++x 执行，x 变为 6
```

1.4 流程控制

switch 表达式（JDK 7+支持String，JDK 14+支持箭头语法）：

```
// 传统写法
switch (score / 10) {
    case 10:
    case 9:
        grade = 'A';
        break;           // 漏写 break 会发生"穿透"
    case 8:
        grade = 'B';
        break;
    default:
        grade = 'F';
}
```

二、数组

2.1 数组的声明与初始化

```
// 声明方式（两种皆可）
int[] arr1;           // 推荐
int arr2[];           // C 风格，不推荐

// 静态初始化（声明时直接赋值）
int[] a = new int[]{1, 2, 3}; // 完整写法
int[] b = {1, 2, 3};           // 简化写法（声明和初始化必须在同一行）

// 动态初始化（指定长度，系统赋默认值）
int[] c = new int[5];           // 元素均为 0
```

2.2 数组的常用操作

```
int[] arr = {3, 1, 4, 1, 5};

// 遍历（普通 for）
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);    // arr.length 不是 length()
}

// 遍历（增强 for / foreach）
for (int n : arr) {
    System.out.println(n);
}

// 数组拷贝
int[] copy = Arrays.copyOf(arr, arr.length);    // 最常用
int[] copy2 = new int[arr.length];
System.arraycopy(arr, 0, copy2, 0, arr.length); // 底层方法，效率高

// 排序
Arrays.sort(arr);

// 转字符串
System.out.println(Arrays.toString(arr));
```

易错点：

- 数组下标从 0 开始，范围是 [0, length-1]
- 访问越界抛 `ArrayIndexOutOfBoundsException`
- `arr.length` 是属性（不是方法），`String.length()` 是方法

2.3 二维数组

```
// 静态初始化
int[][] arr = {{1, 2}, {3, 4, 5}};    // 第二行有 3 个元素

// 动态初始化（不规则数组）
int[][] arr2 = new int[3][];
arr2[0] = new int[2];
arr2[1] = new int[3];
arr2[2] = new int[1];
```

三、面向对象基础

3.1 类与对象

```
public class Student {  
    // 成员变量（属性）  
    private String name;  
    private int age;  
  
    // 构造方法（与类同名，无返回值）  
    public Student() {} // 无参构造  
    public Student(String name, int age) { // 有参构造  
        this.name = name; // this 区分成员变量和局部变量  
        this.age = age;  
    }  
  
    // 成员方法  
    public void study() {  
        System.out.println(name + " 正在学习");  
    }  
  
    // getter / setter  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
}  
  
// 使用  
Student s = new Student("张三", 20); // new + 构造方法创建对象  
s.study();
```

3.2 封装（Encapsulation）

- **private** 修饰成员变量，对外提供 **public** 的 **getter/setter**
- 隐藏内部实现细节，保护数据完整性

3.3 this 关键字

用法	含义
this.成员变量	区分成员变量与局部变量（同名时）
this.成员方法()	调用本类的其他方法
this()	调用本类的其他构造方法（必须在构造方法第一行）

```
public class Person {
    private String name;

    public Person() {
        this("匿名"); // 调用本类有参构造
    }

    public Person(String name) {
        this.name = name;
    }
}
```

3.4 static 关键字

修饰	特点	访问方式
静态变量	被所有对象共享，类加载时分配	类名.变量名（推荐）
静态方法	只能访问静态成员，不能使用 this	类名.方法名
静态代码块	类加载时执行一次，用于初始化	自动执行

```
public class Counter {
    public static int count = 0; // 所有对象共享
    public static final double PI = 3.14159; // 常量（命名全大写）

    public Counter() {
        count++;
    }
}
```

四、面向对象进阶（考试重点）

4.1 继承（Inheritance）

- Java 使用 **单继承**（一个子类只能有一个直接父类），但支持多层继承
- 子类继承父类的**非私有**成员（变量和方法）
- 子类不能继承父类的构造方法

构造方法的调用规则（必考！）：

```
class Father {  
    String name;  
    public Father(String name) {  
        this.name = name;  
    }  
}
```

```
class Son extends Father {  
    public Son(String name) {  
        super(name); // 必须显式调用！因为父类没有无参构造  
    }  
}
```

- 子类构造方法**第一行**必须是 `super(...)` 或 `this(...)` ,不能同时出现
- 若父类有**无参构造**，`super()` 可以省略（系统默认添加）
- 若父类**只有有参构造**，子类必须显式调用 `super(参数)`

4.2 方法重写（Override）

// 规则：方法名、参数列表必须相同；返回值类型可以相同或为子类型
// 访问权限不能更严格（public > protected > 默认 > private）
// 抛出的异常不能更宽泛

```
class Animal {
    public void shout() {
        System.out.println("动物叫");
    }
}

class Dog extends Animal {
    @Override // 建议加上，编译器会检查是否真的重写
    public void shout() {
        System.out.println("汪汪汪");
    }
}
```

overload vs override（高频辨析）：

对比	重载（Overload）	重写（Override）
位置	同一个类中	子父类之间
方法名	相同	相同
参数列表	必须不同	必须相同
返回值	可以不同	必须相同或子类型
访问权限	无关	不能更严格
关键字	无需 @Override	建议 @Override

4.3 super 关键字

super.成员变量; // 访问父类的成员变量（被子类隐藏时）
super.成员方法(); // 调用父类的成员方法
super(参数); // 调用父类的构造方法（必须在子类构造方法第一行）

4.4 final 关键字

修饰位置	效果
final 类	不能被继承（如 String、Math）
final 方法	可以被继承，但不能被重写
final 变量	变为常量，值不可修改（必须初始化）
final 引用	引用指向的对象地址不可变，但对象内容可变

4.5 多态（Polymorphism，必考！）

三个条件：

- 1. 有继承关系
- 2. 子类重写父类方法
- 3. 父类引用指向子类对象

向上转型：

```
Animal a = new Dog();    // 自动转换
a.shout();                // 输出"汪汪汪"（调用子类重写的方法）
// a.wagTail();          // 编译错误！不能调用子类独有方法
```

调用规则（必考点）：

- **成员变量**：编译看左边（父类），运行看左边（父类）
- **成员方法**：编译看左边（父类），运行看右边（子类重写的方法）
- **静态方法**：编译看左边，运行看左边（静态方法无多态）

向下转型 + instanceof：

```
Animal a = new Dog();
if (a instanceof Dog) {    // 先判断，避免 ClassCastException
    Dog d = (Dog) a;
    d.wagTail();
}
```

4.6 抽象类（abstract）

```
// 含有抽象方法的类必须是抽象类
// 抽象类不能实例化（不能 new）
// 子类必须重写所有抽象方法，否则子类也得是抽象类
public abstract class Shape {
    public abstract double getArea(); // 抽象方法，无方法体
}
```

4.7 接口（interface）

```
// 接口用 interface 声明，实现用 implements
// 一个类可以实现多个接口
public interface Flyable {
    // 抽象方法（默认 public abstract）
    void fly();
}

// 实现类必须重写接口中所有抽象方法
public class Bird implements Flyable {
    @Override
    public void fly() {
        System.out.println("鸟飞");
    }
}
```

接口与抽象类的区别：

对比	接口	抽象类
关键字	interface / implements	abstract class / extends
多继承	一个类可实现多个接口	一个类只能继承一个抽象类
构造方法	不能有	可以有
成员变量	只能是 public static final 常量	各种类型均可
设计目的	定义行为规范（能力契约）	代码复用（模板模式）

五、常用类

5.1 String 类

String 不可变性：String 对象一旦创建，内容不可改变。每次修改都会产生新对象。

```
String s1 = "hello";
String s2 = "hello";
String s3 = new String("hello");

System.out.println(s1 == s2);      // true（常量池复用）
System.out.println(s1 == s3);      // false（堆中新对象）
System.out.println(s1.equals(s3)); // true（equals 比较内容）
```

常用方法：

方法	说明
int length()	返回字符串长度
char charAt(int index)	返回指定位置的字符
boolean equals(String s)	比较内容是否相等
boolean equalsIgnoreCase(String s)	忽略大小写比较
int indexOf(String s)	返回子串首次出现的位置
String substring(int begin, int end)	截取子串 [begin, end)
String[] split(String regex)	按正则分割
String replace(char old, char new)	替换字符
boolean contains(CharSequence s)	是否包含子串
String trim()	去除首尾空格
char[] toCharArray()	转为字符数组

StringBuilder（可变字符串，单线程高效）：

```

StringBuilder sb = new StringBuilder();
sb.append("hello");    // 追加
sb.insert(0, "H");     // 插入
sb.reverse();          // 反转
sb.toString();         // 转 String

```

5.2 包装类

基本类型	包装类
int	Integer
char	Character
double	Double
boolean	Boolean

```

// 自动装箱
Integer i = 100;           // 等价于 Integer.valueOf(100)

// 自动拆箱
int n = i;                 // 等价于 i.intValue()

```

Integer 缓存机制（考试易错点！）：

```

Integer a = 127;    // 在缓存范围 [-128, 127] 内，复用对象
Integer b = 127;
System.out.println(a == b);    // true

Integer c = 128;    // 超出缓存范围，创建新对象
Integer d = 128;
System.out.println(c == d);    // false!
System.out.println(c.equals(d)); // true（正确比较方式）

```

结论：包装类比较用 `equals()`，不用 `==`！

5.3 Math 类

```
Math.abs(-5);           // 5, 绝对值
Math.max(3, 5);          // 5, 最大值
Math.min(3, 5);          // 3, 最小值
Math.pow(2, 3);          // 8.0, 幂运算
Math.sqrt(9);            // 3.0, 平方根
Math.random();           // [0.0, 1.0) 随机数
Math.floor(3.9);          // 3.0, 向下取整
Math.ceil(3.1);          // 4.0, 向上取整
Math.round(3.5);         // 4L, 四舍五入
```

典型考题：生成 [min, max] 范围内的随机整数

```
int r = (int)(Math.random() * (max - min + 1)) + min;
// 例如 [10, 20]: (int)(Math.random() * 11) + 10
```

六、异常处理

6.1 异常体系

```
Throwable
├── Error (不可处理, JVM 内部错误)
│   ├── OutOfMemoryError
│   └── StackOverflowError
└── Exception
    ├── 受检异常 (必须处理或抛出)
    │   ├── IOException
    │   ├── FileNotFoundException
    │   └── ClassNotFoundException
    └── 非受检异常 (运行时异常, 可选处理)
        ├── NullPointerException
        ├── ArrayIndexOutOfBoundsException
        ├── ArithmeticException (如除零)
        └── ClassCastException
```

6.2 try-catch-finally

```
try {
    int result = 10 / 0; // 可能抛出异常的代码
} catch (ArithmeticException e) {
    System.out.println("算术异常: " + e.getMessage());
} finally {
    System.out.println("一定会执行（除非 System.exit() 或 JVM 崩溃）");
}
```

try-with-resources (JDK 7+):

```
// 自动关闭实现了 AutoCloseable 的资源
try (BufferedReader br = new BufferedReader(new FileReader("test.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
}
// 无需 finally 手动关闭
```

6.3 throws vs throw

关键字	位置	含义
throws	方法声明末尾	声明该方法可能抛出的异常类型，交给调用者处理
throw	方法体内部	实际抛出一个异常实例

```
public void setAge(int age) throws IllegalArgumentException {
    if (age < 0 || age > 150) {
        throw new IllegalArgumentException("年龄不合法");
    }
}
```

七、集合框架基础

7.1 集合体系

Collection（单列集合）

- └─ List（有序、可重复）
 - └─ ArrayList（数组实现，查询快，增删慢）
 - └─ LinkedList（链表实现，增删快，查询慢）
- └─ Set（无序、不可重复）
 - └─ HashSet（基于 HashMap，无序）
 - └─ TreeSet（红黑树，自动排序）

Map（双列集合，key-value 对）

- └─ HashMap（key 无序，效率高）
- └─ TreeMap（key 自动排序）

7.2 ArrayList 使用

```
// 创建
ArrayList<String> list = new ArrayList<>();

// 增删改查
list.add("A");           // 末尾添加
list.add(0, "B");        // 指定位置插入
list.get(0);             // 获取
list.set(0, "C");        // 修改
list.remove(0);          // 按索引删除
list.remove("A");        // 按对象删除
list.size();             // 元素个数
list.contains("A");       // 是否包含
list.isEmpty();          // 是否为空

// 遍历
for (int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i));
}
for (String s : list) {
    System.out.println(s);
}
```

7.3 HashSet 使用

```
HashSet<Integer> set = new HashSet<>();
set.add(10);
set.add(20);
set.add(10);           // 重复元素不会添加，返回 false

// 遍历（没有顺序）
for (Integer n : set) {
    System.out.println(n);
}
```

7.4 HashMap 使用

```
HashMap<String, Integer> map = new HashMap<>();

// 增删改查
map.put("张三", 85);           // 添加/修改
map.get("张三");               // 获取（不存在返回 null）
map.remove("张三");           // 删除
map.containsKey("张三");       // 是否包含 key
map.containsValue(85);         // 是否包含 value
map.size();                    // 键值对个数

// 遍历（常考）
for (String key : map.keySet()) {
    System.out.println(key + "=" + map.get(key));
}
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + "=" + entry.getValue());
}
```


7.5 Collections 工具类

```
List<Integer> list = new ArrayList<>(Arrays.asList(3, 1, 4, 1, 5));
Collections.sort(list);           // 排序
Collections.reverse(list);        // 反转
Collections.shuffle(list);        // 打乱
Collections.max(list);            // 最大值
Collections.min(list);            // 最小值
Collections.binarySearch(list, 3); // 二分查找（先排序）
```

八、I/O 流

8.1 流的概念与分类

分类标准	类别	说明
方向	输入流 / 输出流	读数据 / 写数据
单位	字节流 / 字符流	字节流处理二进制，字符流处理文本
功能	节点流 / 处理流	节点流直接操作数据源，处理流包装节点流

8.2 字节流

```
// 字节输入流：FileInputStream
// 字节输出流：FileOutputStream

// 读文件（推荐写法）
try (FileInputStream fis = new FileInputStream("input.txt")) {
    int data;
    while ((data = fis.read()) != -1) {
        System.out.print((char) data);
    }
} catch (IOException e) {
    e.printStackTrace();
}

// 读文件（缓冲区方式，效率更高）
try (FileInputStream fis = new FileInputStream("input.txt")) {
    byte[] buffer = new byte[1024];
    int len;
    while ((len = fis.read(buffer)) != -1) {
        // buffer[0..len-1] 为读取的数据
    }
} catch (IOException e) {
    e.printStackTrace();
}

// 复制文件
try (FileInputStream fis = new FileInputStream("source.jpg");
    FileOutputStream fos = new FileOutputStream("dest.jpg")) {
    byte[] buffer = new byte[1024];
    int len;
    while ((len = fis.read(buffer)) != -1) {
        fos.write(buffer, 0, len);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

8.3 字符流

```
// 字符输入流：FileReader
// 字符输出流：FileWriter

// 读文本文件
try (FileReader fr = new FileReader("input.txt")) {
    int data;
    while ((data = fr.read()) != -1) {
        System.out.print((char) data);
    }
} catch (IOException e) {
    e.printStackTrace();
}

// 写文本文件
try (FileWriter fw = new FileWriter("output.txt")) {
    fw.write("Hello World");
    fw.write("\r\n");        // 换行
    fw.write("第二行");
} catch (IOException e) {
    e.printStackTrace();
}
```

8.4 缓冲流（推荐，效率高）

```
// 缓冲字符流：BufferedReader / BufferedWriter

// 逐行读取（最常用）
try (BufferedReader br = new BufferedReader(new FileReader("input.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {    // readLine() 不含换行符
        System.out.println(line);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

8.5 File 类

```
File file = new File("D:/test/hello.txt");

file.exists();           // 是否存在
file.isFile();           // 是否为文件
file.isDirectory();     // 是否为目录
file.getName();          // 文件名: hello.txt
file.length();           // 文件大小 (字节)
file.getAbsolutePath(); // 绝对路径

// 目录操作
File dir = new File("D:/test");
String[] names = dir.list();           // 文件名列表
File[] files = dir.listFiles();       // File 数组
```

九、Object 类常用方法

Object 是所有类的根父类。

9.1 equals() 与 hashCode()

- **规则：** equals() 相等 → hashCode() 必须相等； hashCode() 相等 → equals() 不一定相等
- 重写 equals() **必须同时重写** hashCode() ，否则 HashSet/HashMap 会行为异常

```
public class Student {
    private String id;

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        Student s = (Student) o;
        return Objects.equals(id, s.id);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}
```

9.2 toString()

- 默认返回 类名@十六进制哈希码 （如 Student@3f99bd52 ）
- 通常重写以输出对象的属性信息

十、综合易错点与常见考题

10.1 经典辨析题

错误说法	正确说法
final 方法不能被继承	final 方法 可以被继承 ，但 不能被重写
static 方法可以被重写	static 方法只能被 隐藏 ，调用哪个取决于引用类型
多态调用成员变量看右边（实际对象）	多态调用 成员变量看左边 （引用类型）
多态调用方法也看左边	多态调用 方法看右边 （实际对象重写的方法）
枚举可以继承其他类	枚举已继承 java.lang.Enum ，不能再继承其他类
子类不会继承父类的私有变量	子类 会继承 私有变量，但 无法直接访问

错误说法	正确说法
finally 一定执行	System.exit() 或 JVM 崩溃时 不执行
接口中 default 方法重写时保留 default	重写必须去掉 default 关键字
ArrayList 增删总是很慢	尾部 添加很快 $O(1)$ 摊销， 中间 插入/删除才慢 $O(n)$
== 可以比较 String 内容	String 用 equals() 比较内容，== 比较引用地址

10.2 高频代码填空题

(1) 统计字符串中每个字符出现次数：

```
String str = "hello";
Map<Character, Integer> map = new HashMap<>();
for (char c : str.toCharArray()) {
    map.put(c, map.getOrDefault(c, 0) + 1);
}
```

(2) 文件复制：

```
try (FileInputStream fis = new FileInputStream("src.txt");
    FileOutputStream fos = new FileOutputStream("dst.txt")) {
    byte[] buf = new byte[1024];
    int len;
    while ((len = fis.read(buf)) != -1) {
        fos.write(buf, 0, len);
    }
}
```

(3) 冒泡排序：

```
for (int i = 0; i < arr.length - 1; i++) {
    for (int j = 0; j < arr.length - 1 - i; j++) {
        if (arr[j] > arr[j + 1]) {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
```

10.3 重点复习建议

优先等级	知识点	典型分值
☆☆☆	基本语法（数据类型、运算符、流程控制）	15-20%
☆☆☆	面向对象（构造方法、继承、多态、重写）	25-30%
☆☆☆	String 类（常用方法、比较）	10-15%
☆☆☆	异常处理（try-catch-finally、throws/throw）	8-10%
☆☆	集合（ArrayList、HashMap 基本使用）	10-15%
☆☆	I/O 流（字节流、字符流、BufferedReader）	10-12%
☆	数组、包装类、Math、static/final	8-10%